

LAPORAN PEMOGRAMAN BERORIENTASI OBJEK
PEKAN 4 EKSPANSI PRODUK KEUANGAN (INHERITANCE)



Disusun Oleh :

DIKA GIOWANDA

NIM:

2311533025

Dosen Pengampu :

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Muhammad Dawi Syauqi

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG 2026

A. TAHAPAN PELAKSANAAN PRAKTIKUM

a. Teori Java Modul 4

Seiring berkembangnya sistem perbankan, bank tidak hanya memiliki satu jenis rekening. Ada "Rekening Tabungan" yang mana akan mendapat bunga bulanan dan "Rekening Giro" yang memiliki fasilitas pinjaman atau overdraft. Jika kita membuat class baru dari nol untuk masing-masing entitas tersebut, kita akan melanggar prinsip DRY (Don't Repeat Yourself) karena menduplikasi banyak kode logika PIN dan mutasi.

1. Konsep Pewarisan (Inheritance): Menurut Deitel dan Deitel (2017), pewarisan adalah bentuk software reuse di mana class baru diciptakan dengan menggunakan atribut dan behavior dari class yang sudah ada, lalu menambahkan kapabilitas baru. Hubungan ini dikenal sebagai relasi "Is-A" contohnya Tabungan is-a Rekening.
2. Penggunaan Keyword extends dan super : Di Java, subclass mewarisi kelas induk menggunakan kata kunci extends. Sementara itu, kata kunci super berfungsi sebagai jembatan langsung ke kelas induk:
 - `super()` digunakan di dalam konstruktor kelas anak untuk mengeksekusi inisialisasi konstruktor induk. Pemanggilan ini wajib berada di baris paling pertama di dalam konstruktor anak (Oracle, 2023).
 - `super.namaMethod()` digunakan jika kelas anak perlu memanggil metode asli milik induk yang mungkin telah dimodifikasi (di-override).
3. Pentingnya protected: Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.

4. Upcasting dan Downcasting: Karena adanya relasi Is-A, Java mengizinkan konversi tipe referensi antar hierarki kelas (Eckel, 2006):

- Upcasting (Implisit): Menyimpan objek kelas anak ke dalam variabel referensi kelas induk contohnya Rekening akun = new RekeningTabungan(...). Proses ini otomatis dan aman. Fungsinya sangat krusial agar kita bisa menyimpan berbagai jenis produk rekening ke dalam satu koleksi yang seragam seperti ArrayList tipe Rekening.
- Downcasting (Eksplisit): Memaksa referensi kelas induk kembali menjadi kelas anak spesifik, contoh RekeningTabungan tab = (RekeningTabungan) akun;. Proses ini rawan memicu error ClassCastException saat program berjalan jika objek aslinya ternyata bukan Tabungan. Oleh karena itu, downcasting wajib didahului oleh pengecekan tipe menggunakan operator instanceof.

b. Langkah Langkah Menerapkan Ekspansi Produk Keuangan (Inheritance)

1. Memindahkan atau menyalin class Transaksi dari package sebelumnya ke package baru.

Buat package baru, salin kode class transaksi yang ada di package praktikum sebelumnya, lalu tempel ke class transaksi pada package yang baru.

2. Memastikan akses modifier di class Rekening

Masuk ke class Rekening, mengubah atribut saldo dan riwayat transaksi menjadi protected, tujuannya agar kelas turunan (subclass atau child class) dari kelas Rekening (seperti RekeningTabungan atau RekeningGiro) dapat mengakses, membaca, dan mengubah data saldo serta riwayat transaksi secara langsung. Sementara untuk atribut nomorRekening, namaPemilik, dan pin tetap private .

```

1 package Pekan4_2511533025;
2 import java.util.ArrayList;
3
4 public class Rekening {
5     // 1. Atribut private
6     private String nomorRekening;
7     private String namaPemilik;
8     private String pin; // Data sensitif!
9
10    // Gunakan protected agar Subclass bisa mengaksesnya langsung
11    protected double saldo;
12    protected ArrayList<Transaksi> riwayatTransaksi;
13

```

3. Membuat subclass baru yaitu class RekeningTabungan
Menambahkan fitur baru yaitu suku Bunga bulanan

```

1 package Pekan4_2511533025;
2
3 public class RekeningTabungan extends Rekening {
4     // Atribut spesifik yang hanya dimiliki oleh Tabungan
5     private double sukuBunga;
6
7     // Constructor Subclass
8     public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
9         // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
10        super(nomor, nama, saldoAwal, pinAwal);
11        this.sukuBunga = sukuBunga;
12    }
13
14    public void tambahBungaAkhirBulan() {
15        // Menghitung bunga
16        // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
17        // Karena atribut 'saldo' di class Rekening menggunakan modifier 'protected'
18        double nominalBunga = saldo * (sukuBunga / 100);
19        saldo += nominalBunga;
20
21        // Mencatat riwayat transaksi
22        String idTrx = "TRX-B-" + System.currentTimeMillis();
23        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
24
25        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
26    }
27 }
28

```

4. Membuat subclass baru yaitu Rekening Giro
Menambahkan fitur baru yang biasanya digunakan oleh pembisnis yaitu batas overdraft(limit pinjaman).

```

1 package Pekan4_2511533025;
2
3 public class RekeningGiro extends Rekening {
4     private double batasOverdraft;
5
6     public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
7         // Memanggil inisialisasi dasar dari Superclass
8         super(nomor, nama, saldoAwal, pinAwal);
9         this.batasOverdraft = batasOverdraft;
10    }
11
12    // Getter khusus Giro
13    public double getBatasOverdraft() {
14        return batasOverdraft;
15    }
16
17    // (Catatan: Penarikan hingga limit overdraft akan kita selesaikan di Modul 5)
18 }
19

```

5. Menambahkan Menu baru pada class mainPerbankan

Pada praktikum kali ini, perlu memperbarui class mainPerbankan supaya subclass seperti RekeningTabungan dan class RekeningGiro dapat dijalankan. Karena tugas praktikum ke empat ini merujuk ke class mainPerbankan , jadi sekaligus untuk penjelasan laporan tugas praktikum

digabung dengan laporan praktikum. Jadi sekaligus digabungkan karena tugas terhubung dengan pembaruan class mainPerbankan.

- Menambahkan pilihan menu yang awalnya 7 menjadi 8, yaitu menu ke 7 Simulasi Akhir Bulan

```

22      System.out.println("Menu Utama:");
23      System.out.println("1. Buka Rekening Baru");
24      System.out.println("2. Setor Tunai");
25      System.out.println("3. Tarik Tunai");
26      System.out.println("4. Cek Informasi Rekening");
27      System.out.println("5. Ganti Akun (Pilih Akun)");
28      System.out.println("6. Tampilkan Riwayat Transaksi");
29      System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
30      System.out.println("0. Keluar");
31      System.out.print("Pilih menu: ");
32

```

- Perubahan case 1 (Pembuatan Rekening Baru & Upcasting), yaitu menambahkan submenu pilihan jenis produk (1. Tabungan Umum / 2. Giro Bisnis). Membuat objek subclass RekeningTabungan atau RekeningGiro lalu menyimpannya ke dalam ArrayList<Rekening> (Upcasting).

```

37 // Modifikasi Case 1: Buka Rekening Baru (Tugas 1)
38 System.out.print("Masukkan No. Rekening: ");
39 String no = scanner.nextLine();
40 System.out.print("Masukkan Nama Pemilik: ");
41 String nama = scanner.nextLine();
42 System.out.print("Masukkan Saldo Awal: ");
43 double saldoAwal = scanner.nextDouble();
44 scanner.nextLine(); // Clear buffer
45 System.out.print("Buat PIN (6 digit): ");
46 String pin = scanner.nextLine();
47
48 // 1. Opsi Produk
49 System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
50 System.out.print("Pilihan Anda: ");
51 int jenisProduk = scanner.nextInt();
52 scanner.nextLine(); // Clear buffer
53
54 Rekening rekBaru = null;
55
56 if (jenisProduk == 1) {
57     // 2. Instansiasi RekeningTabungan
58     System.out.print("Masukkan Suku Bunga (%): ");
59     double sukuBunga = scanner.nextDouble();
60     scanner.nextLine();
61     rekBaru = new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga);
62 } else if (jenisProduk == 2) {
63     // 3. Instansiasi RekeningGiro
64     System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
65     double batasOverdraft = scanner.nextDouble();
66     scanner.nextLine();
67     rekBaru = new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft);
68 } else {
69     System.out.println("Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
70     break;
71 }
72
73 // 4. Upcasting: Menyimpan ke daftarRekening bertipe ArrayList<Rekening>
74 daftarRekening.add(rekBaru);
75 akunAktif = rekBaru; // Otomatis dijadikan akun aktif
76 break;
77
78

```

- Perubahan Tipe Iterasi pada case 6, yaitu yang awalnya menggunakan Object trx pada perulangan for-each, diganti menggunakan tipe spesifik Transaksi trx.

```

152      System.out.println("---- RIWAYAT TRANSAKSI ----");
153      for (Transaksi trx : akunAktif.getRiwayatTransaksi()) {
154          System.out.println(trx);
155      }
156

```

- Penambahan case 7, yaitu menambahkan opsi menu ke-7. Di dalam case 7, dilakukan pengecekan apakah akunAktif merupakan objek

dari RekeningTabungan menggunakan keyword instanceof, kemudian dilakukan Downcasting (RekeningTabungan) akunAktif untuk memanggil method tambahBungaAkhirBulan().

```

164 // Simulasi Waktu Akhir Bulan (Tugas 2)
165 // (akunAktif == null) {
166 //     System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
167 } else {
168 // 2. Memeriksa tipe objek dengan instanceof
169 // if (akunAktif instanceof RekeningTabungan) {
170 //     // 3. Downcasting ke tipe RekeningTabungan & memanggil method
171 //     RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
172 //     tabungan.tambahBungaAkhirBulan();
173 // } else {
174 //     // 4. Pesan penolakan jika bukan RekeningTabungan
175 //     System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
176 // }
177 }
178 break;
179

```

c. Penjelasan Kode Program Java

a. Class Transaksi

1. Deklarasi Class dan Pembentukan Atribut

Menggunakan Access Modifier private pada atribut idTransaksi, jenis, dan nominal. Menjaga atribut agar tidak dapat diakses atau dimanipulasi secara langsung dari luar Class (mencegah perubahan data ilegal).

```

1 package Pekan4_2511533025;
2
3 public class Transaksi {
4     // 1. Mengubah semua atribut menjadi private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8

```

2. Inisialisasi Objek (Penerapan Constructor)

Memiliki Constructor berparameter dengan modifier public untuk menginisialisasi nilai awal object saat dibuat. Menggunakan kata kunci this untuk membedakan antara variabel atribut class dengan parameter yang dimasukkan.

```

9 // Constructor
10 public Transaksi(String id, String jenis, double nominal) {
11     this.idTransaksi = id;
12     this.jenis = jenis;
13     this.nominal = nominal;
14 }
15

```

3. Penerapan Getter / Read-Only

Mengimplementasikan Method Getter ber-modifier public untuk memfasilitasi akses pembacaan nilai variabel private dari luar class. Tidak menyediakan Setter sehingga nilai object bersifat Read-Only (tidak bisa diubah nilainya setelah object dibuat).

```
16 // 2. Hanya menyediakan Getter (Read-Only)
17 public String getIdTransaksi() { return idTransaksi; }
18 public String getJenis() { return jenis; }
19 public double getNominal() { return nominal; }
20
```

4. Menampilkan Informasi Data (Penerapan Method / Behavior)

Method cetakDetail() bertindak sebagai Behavior untuk menampilkan seluruh informasi data object ke konsol secara langsung. Method @Override toString() mengembalikan representasi teks (String) dari object agar mudah dicetak atau digabungkan dalam output lain.

```
21 public void cetakDetail() {
22     System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
23 }
24
25 @Override
26 public String toString() {
27     return "ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal;
28 }
29
```

5. Kode Secara Lengkap

```
package Pekan4_2511533025;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }
}
```

```

        public void cetakDetail() {
            System.out.println("ID: " + idTransaksi + " |
Jenis: " + jenis + " | Nominal: Rp" + nominal);
        }

        @Override
        public String toString() {
            return "ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nominal;
        }
    }
}

```

b. Class Rekening

1. Inisialisasi Class dan Mengunci Atribut dan Pemberian Hak Akses

Pada class Rekening memanggil pustaka Java ArrayList agar bisa menggunakannya untuk membuat daftar dinamis. Pada baris ke empat mendeklarasikan kelas utama bernama Rekening dengan akses public. Kemudian menambahkan hak akses private (enkapsulasi) sehingga tidak bisa diakses langsung dari luar kelas Rekening dan penambahan hak akses protected agar dapat diakses langsung oleh kelas turunan (subclass) dari kelas Rekening..

```

1 package Pekan4_2511533025;|
2 import java.util.ArrayList;
3
4 public class Rekening {
5     // 1. Atribut private
6     private String nomorRekening;
7     private String namaPemilik;
8     private String pin; // Data sensitif!
9
10    // Gunakan protected agar Subclass bisa mengaksesnya langsung
11    protected double saldo;
12    protected ArrayList<Transaksi> riwayatTransaksi;
13

```

2. Bagian Modifikasi Constructor dan Validasi PIN Awal

Constructor menerima masukan tambahan berupa pinAwal. Sebelum disimpan ke atribut this.pin, dilakukan validasi kondisi: jika panjang PIN adalah 6 digit (length() == 6), PIN akan disimpan. Jika tidak, sistem mencetak peringatan dan menetapkan PIN default ("123456"). ArrayList riwayat transaksi juga diinisialisasi di sini untuk mencegah error NullPointerException.

```

14 // 2. Modifikasi Constructor untuk menerima PIN awal
15 public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
16     this.nomorRekening = nomor;
17     this.namaPemilik = nama;
18     this.saldo = saldoAwal;
19
20     // Validasi PIN di dalam Constructor
21     if (pinAwal.length() == 6) {
22         this.pin = pinAwal;
23     } else {
24         System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
25         this.pin = "123456";
26     }
27     // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
28     this.riwayatTransaksi = new ArrayList<>();
29
30     System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
31 }
32

```

3. Bagian Pembuatan Method Getter dan Method Otentikasi

Karena atribut bersifat private, dibuatkan method khusus untuk membaca data dan memverifikasi keamanan:

- `getNomorRekening()` & `getNamaPemilik()` : Method Getter yang memberikan akses baca publik terhadap nomor rekening dan nama pemilik tanpa memberikan akses untuk mengeditnya.
- `otentikasi(String inputPin)` : Method pemeriksaan keamanan internal yang mencocokkan input PIN pengguna dengan PIN milik objek menggunakan `.equals()`. Menghasilkan nilai boolean `true` atau `false`.

```

33
34 // 3. Getter untuk atribut yang diizinkan dibaca publik
35 public String getNomorRekening() { return nomorRekening; }
36 public String getNamaPemilik() { return namaPemilik; }
37
38 // Getter untuk mengakses riwayat transaksi dari luar kelas
39 public ArrayList<Transaksi> getRiwayatTransaksi() {
40     return this.riwayatTransaksi;
41 }
42
43 // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
44 public boolean otentikasi(String inputPin) {
45     return this.pin.equals(inputPin);
46 }
47

```

4. Method Setor Tunai (`setorTunai`)

Method ini menangani proses penambahan saldo dengan alur logika berikut:

- Validasi: Mengurangi `System.currentTimeMillis()` bernilai positif (> 0).
- Update Saldo: Menambahkan nilai nominal ke variabel saldo.

- Pencatatan Transaksi: Membuat objek Transaksi baru dengan ID berbasis waktu (timestamp `System.currentTimeMillis()`) dan tipe "Kredit", lalu memasukkannya ke dalam list riwayatTransaksi dengan metode `.add()`.

```

48 public void setorTunai(double nominal) {
49     if (nominal > 0) {
50         saldo += nominal;
51
52         // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
53         String idTrx = "TRX-S-" + System.currentTimeMillis();
54         Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
55         riwayatTransaksi.add(trxBaru);
56         System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
57     } else {
58         System.out.println("Gagal: Nominal setor harus lebih dari 0!");
59     }
60 }

```

5. Method Tarik Tunai (tarikTunai)

Method ini mengelola proses pengurangan saldo dengan menggunakan percabangan/pemeriksaan bertingkat (conditional statement):

- Kondisi 1: Memastikan nominal penarikan minimal Rp10.000.
- Kondisi 2: Memastikan saldo yang tersedia mencukupi nominal yang diminta penarikan.
- Eksekusi: Jika lolos dari kedua validasi di atas, saldo akan dikurangi, objek Transaksi baru dibuat dengan jenis "Debit", lalu dicatat ke dalam riwayatTransaksi.

```

61 public void tarikTunai(double nominal) {
62     if (nominal < 10000) {
63         System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
64     } else if (nominal > saldo) {
65         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
66     } else {
67         saldo -= nominal;
68
69         // Merekam riwayat transaksi tarik tunai (Debit)
70         String idTrx = "TRX-T-" + System.currentTimeMillis();
71         Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
72         riwayatTransaksi.add(trxBaru);
73         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
74     }
75 }

```

6. Method Cek Informasi (cekInformasi)

Method pendukung yang bertugas mencetak/menampilkan ringkasan status rekening terkini ke konsol, meliputi nomor rekening, nama pemilik, dan saldo akhir.

```

79 public void cekInformasi() {
80     System.out.println("--- INFO REKENING ---");
81     System.out.println("No. Rekening : " + nomorRekening);
82     System.out.println("Nama Pemilik : " + namaPemilik);
83     System.out.println("Saldo Akhir : Rp" + saldo);
84     System.out.println("-----");
85 }
86 }

```

7. Kode Secara Lengkap

```

package Pekan4_2511533025;
import java.util.ArrayList;

public class Rekening {
    // 1. Atribut private
    private String nomorRekening;
    private String namaPemilik;
    private String pin; // Data sensitif!

    // Gunakan protected agar Subclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // Constructor
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        // Inisialisasi ArrayList
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    // Getter
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    public ArrayList<Transaksi> getRiwayatTransaksi() {

```

```

        return this.riwayatTransaksi;
    }

    // Otentikasi Internal
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;

            String idTrx = "TRX-S-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx,
"Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal
+ " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus
lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal
nominal penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo
tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx,
"Debit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal
+ " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("---- INFO REKENING ----");
        System.out.println("No. Rekening : " +
nomorRekening);
        System.out.println("Nama Pemilik : " +
namaPemilik);
        System.out.println("Saldo Akhir : Rp" + saldo);
        System.out.println("-----");
    }
}

```

}

c. Class MainPerbankan

1. Inisialisasi Class dan Struktur Data Utama dan Menggunakan Method Getter

Pada method main, dibuat beberapa variabel awal:

- Scanner scanner: Mengambil input interaktif dari pengguna via konsol.
- ArrayList<Rekening> daftarRekening: Menyimpan seluruh koleksi rekening yang terdaftar di sistem.
- Rekening akunAktif: Pointer/referensi yang menunjukkan rekening mana yang sedang diakses saat ini (bernilai null jika belum ada yang dipilih).

```
1 package Pekan4_2511533025;  
2  
3 import java.util.ArrayList;  
4 import java.util.Scanner;  
5 public class MainPerbankan {  
6     public static void main(String[] args) {  
7         Scanner scanner = new Scanner(System.in);  
8         ArrayList<Rekening> daftarRekening = new ArrayList<>();  
9         Rekening akunAktif = null;  
10        int pilihan;  
--
```

2. Perulangan Menu Utama (Do-While & Switch-Case) dan Inisialisasi Class dan Menggunakan Method

Header Menu & Cek Status Akun (Baris 12–20):

Menampilkan judul program === SISTEM PERBANKAN MINI (MULTI-AKUN) ===.

Memeriksa variabel akunAktif. Jika nilainya null, sistem mencetak "Akun Aktif : Tidak ada akun yang dipilih". Jika ada akun yang terpilih, sistem menampilkan nama pemilik dan nomor rekeningnya menggunakan method getNamaPemilik() dan getNomorRekening().

Daftar Pilihan Menu (Baris 21–30):

Menampilkan 8 opsi navigasi program (Buka Rekening Baru, Setor Tunai, Tarik Tunai, Cek Informasi, Ganti Akun, Riwayat Transaksi, Simulasi Akhir Bulan, dan Keluar).

Input Pengguna (Baris 32–33):

`pilihan = scanner.nextInt();` mengambil input angka dari pengguna untuk memilih menu.

`scanner.nextLine();` membersihkan sisa karakter newline (enter) dari buffer agar input berbentuk teks selanjutnya tidak terlewat (Clear buffer).

```
12 do {
13     System.out.println("\n== SISTEM PERBANKAN MINI (MULTI-AKUN) ==");
14     System.out.println("-----");
15     if (akunAktif == null) {
16         System.out.println("Akun Aktif : Tidak ada akun yang dipilih");
17     } else {
18         System.out.println("Akun Aktif : " + akunAktif.getNamaPemilik() + " (" + akunAktif.getNomorRekening() + ")");
19     }
20     System.out.println("-----");
21     System.out.println("Menu Utama:");
22     System.out.println("1. Buka Rekening Baru");
23     System.out.println("2. Setor Tunai");
24     System.out.println("3. Tarik Tunai");
25     System.out.println("4. Cek Informasi Rekening");
26     System.out.println("5. Ganti Akun (Pilih Akun)");
27     System.out.println("6. Tampilkan Riwayat Transaksi");
28     System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
29     System.out.println("8. Keluar");
30     System.out.print("Pilih menu: ");
31
32     pilihan = scanner.nextInt();
33     scanner.nextLine(); // Clear buffer
34 }
```

3. Penjelasan Logika Setiap Menu(Switch Case)

a. Case 1: Buka Rekening Baru

Input Data Nasabah (Baris 38–46):

Meminta input dasar berupa nomor rrekBaru, nama pemilik, saldo awal, dan PIN dari pengguna menggunakan Scanner.

`scanner.nextLine()` di baris 44 digunakan untuk membersihkan buffer dari sisa baris baru setelah `nextDouble()`.

Pemilihan Jenis Produk (Baris 48–52):

Pengguna diminta memilih produk: 1 untuk Tabungan Umum atau 2 untuk Giro Bisnis.

Constructor & Instansiasi (Baris 54–71):

`Rekening rekBaru = null;` disiapkan sebagai variabel referensi kelas induk (Rekening).

Opsi 1 (RekeningTabungan): Meminta input suku bunga, lalu memanggil constructor `new RekeningTabungan(...)`.

Opsi 2 (RekeningGiro): Meminta input batas overdraft (limit pinjaman), lalu memanggil constructor new RekeningGiro(...).

Jika pilihan tidak valid, program menampilkan pesan kesalahan dan menghentikan proses pembuatan rekening dengan break;.

Upcasting & Penyimpanan Struktur Data (Baris 73–75):

Upcasting: Objek RekeningTabungan atau RekeningGiro disimpan ke dalam daftarRekening bertipe ArrayList<Rekening>. Kelas turunan secara otomatis dikonversi menjadi tipe kelas induknya (Rekening).

Pengeset Akun Aktif: akunAktif = rekBaru; secara otomatis menetapkan rekening yang baru dibuat sebagai akun yang sedang aktif digunakan di dalam sistem.

```
36 // Modifikasi Case 1: Buka Rekening Baru (Tugas 1)
37 System.out.print("Masukkan No. Rekening: ");
38 String no = scanner.nextLine();
39 System.out.print("Masukkan Nama Pemilik: ");
40 String nama = scanner.nextLine();
41 System.out.print("Masukkan Saldo Awal: ");
42 double saldoAwal = scanner.nextDouble();
43 scanner.nextLine(); // Clear buffer
44 System.out.print("Buat PIN (6 digit): ");
45 String pin = scanner.nextLine();
46
47 // 1. Opsi Produk
48 System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
49 int jenisProduk = scanner.nextInt();
50 scanner.nextLine(); // Clear buffer
51
52 Rekening rekBaru = null;
53
54 if (jenisProduk == 1) {
55     // 2. Instansiasi RekeningTabungan
56     System.out.print("Masukkan Suku Bunga (%): ");
57     double sukuBunga = scanner.nextDouble();
58     scanner.nextLine();
59     rekBaru = new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga);
60 } else if (jenisProduk == 2) {
61     // 3. Instansiasi RekeningGiro
62     System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
63     double batasOverdraft = scanner.nextDouble();
64     scanner.nextLine();
65     rekBaru = new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft);
66 } else {
67     System.out.println("Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
68     break;
69 }
70
71 // 4. Upcasting: Menyimpan ke daftarRekening bertipe ArrayList<Rekening>
72 daftarRekening.add(rekBaru);
73 akunAktif = rekBaru; // Otomatis jadikan akun aktif
74 break;
75
76
77
```

b. Case 2 & 3: Setor & Tarik Tunai

Pemeriksaan Akun Aktif (Baris 79–81 & 89–91)

Program mengecek apakah akunAktif bernilai null. Jika pengguna belum membuat atau memilih akun, sistem mencetak pesan peringatan "Peringatan: Pilih atau buat akun terlebih dahulu!" dan membatalkan transaksi.

Input dan Proses Setor Tunai (Baris 82–85)

Jika akun sudah aktif, program meminta input nominal setoran melalui `scanner.nextDouble()` lalu menyimpannya ke variabel `setor`. Setelah itu, program memanggil method `akunAktif.setorTunai(setor)` untuk menambahkan saldo.

Input dan Otentikasi PIN (Baris 92–94)

Pada menu tarik tunai, program meminta pengguna memasukkan PIN terlebih dahulu lewat `scanner.nextLine()`. String PIN tersebut langsung diuji menggunakan method `akunAktif.otentikasi(inputPin)` untuk memastikan keamanannya.

Proses Tarik Tunai dan Penanganan PIN Salah (Baris 95–100)

Jika hasil otentikasi PIN bernilai benar (`true`), program meminta input nominal penarikan tarik dan menjalankan method `akunAktif.tarikTunai(tarik)`. Jika PIN salah, sistem akan menampilkan pesan "Transaksi Gagal: PIN salah!"..

```
78 case 2:
79     if (akunAktif == null) {
80         System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
81     } else {
82         System.out.print("Masukkan nominal setor tunai: ");
83         double setor = scanner.nextDouble();
84         akunAktif.setorTunai(setor);
85     }
86     break;
87
88 case 3:
89     if (akunAktif == null) {
90         System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
91     } else {
92         System.out.print("Masukkan PIN Anda: ");
93         String inputPin = scanner.nextLine();
94         if (akunAktif.otentikasi(inputPin)) {
95             System.out.print("Masukkan nominal tarik tunai: ");
96             double tarik = scanner.nextDouble();
97             akunAktif.tarikTunai(tarik);
98         } else {
99             System.out.println("Transaksi Gagal: PIN salah!");
100         }
101     }
102     break;
```

c. Case 4: Cek Informasi Rekening

Memanggil method `cekInformasi()` milik `akunAktif` untuk melihat saldo dan identitas pemilik.

```
104 case 4:
105     if (akunAktif == null) {
106         System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
107     } else {
108         akunAktif.cekInformasi();
109     }
110     break;
```

d. Case 5: Ganti Akun (Pilih Akun)

Menampilkan daftar seluruh rekening yang ada di dalam daftarRekening menggunakan perulangan for. Pengguna memilih nomor urut untuk mengubah referensi akunAktif. Saat memilih dan beralih ke rekening lain dari daftar, pengguna wajib memasukkan PIN dari akun target tersebut (target.otentikasi(pinAkses)). Hal ini memastikan bahwa nasabah tidak bisa sembarangan beralih ke akun milik orang lain tanpa otorisasi.

```

112- case 5:
113-     if (daftarRekening.isEmpty()) {
114-         System.out.println("Belum ada rekening yang terdaftar.");
115-     } else {
116-         System.out.println("— DAFTAR AKUN —");
117-         for (int i = 0; i < daftarRekening.size(); i++) {
118-             System.out.println((i + 1) + ". " + daftarRekening.get(i).getNamaPemilik() + " (" + daftarRekening.get(i).getNomorRekening() + ")");
119-         }
120-         System.out.print("Pilih nomor akun: ");
121-         int indeks = scanner.nextInt() - 1;
122-         scanner.nextLine(); // Clear buffer
123-
124-         if (indeks >= 0 && indeks < daftarRekening.size()) {
125-             Rekening target = daftarRekening.get(indeks);
126-             System.out.print("Masukkan PIN akun target: ");
127-             String pinAkses = scanner.nextLine();
128-
129-             if (target.otentikasi(pinAkses)) {
130-                 akunAktif = target;
131-                 System.out.println("Berhasil beralih ke akun " + akunAktif.getNamaPemilik());
132-             } else {
133-                 System.out.println("Gagal beralih: PIN salah!");
134-             }
135-         } else {
136-             System.out.println("Pilihan tidak valid.");
137-         }
138-     }
139-     break;
140-

```

e. Case 6: Tampilkan Riwayat Transaksi

Pemeriksaan Akun Aktif (Baris 142–144):

Memeriksa kondisi apakah akunAktif == null.

Jika belum ada akun yang dibuat atau dipilih, sistem mencetak pesan peringatan "Peringatan: Pilih atau buat akun terlebih dahulu!" dan proses dibatalkan.

Input dan Otentikasi PIN (Baris 145–147):

Meminta pengguna memasukkan PIN menggunakan scanner.nextLine() yang disimpan ke variabel inputPin.

Menguji kebenaran PIN dengan memanggil method akunAktif.otentikasi(inputPin).

Pemeriksaan dan Menampilkan Riwayat Transaksi (Baris 148–156):

Jika PIN Benar:

Memeriksa apakah daftar riwayat kosong menggunakan akunAktif.getRiwayatTransaksi().isEmpty().

Jika kosong, mencetak pesan "Belum ada riwayat transaksi pada akun ini."

Jika ada transaksi, program melakukan perulangan for-each (for (Transaksi trx : akunAktif.getRiwayatTransaksi())) untuk mencetak setiap data transaksi (trx) ke layar.

Penanganan PIN Salah (Baris 157–159):

Jika PIN Salah: Menjalankan blok else dan mencetak pesan kesalahan "Akses Ditolak: PIN salah!"..

```
114 case 6:
115     if (akunAktif == null) {
116         System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
117     } else {
118         System.out.print("Masukkan PIN Anda: ");
119         String inputPin = scanner.nextLine();
120         if (akunAktif.otentikasi(inputPin)) {
121             if (akunAktif.getRiwayatTransaksi().isEmpty()) { // Menggunakan getter agar sesuai encapsulation
122                 System.out.println("Belum ada riwayat transaksi pada akun ini.");
123             } else {
124                 System.out.println("----- RIWAYAT TRANSAKSI -----");
125                 for (Object trx : akunAktif.getRiwayatTransaksi()) { // Atau gunakan tipe objek Transaksi jika ada kelasnya
126                     System.out.println(trx); // Memanggil representasi transaksi
127                 }
128                 System.out.println("-----");
129             }
130         } else {
131             System.out.println("Akses Ditolak: PIN salah!");
132         }
133     }
134     break;
```

f. Case 7: Simulasi Bunga Akhir Bulan

Pemeriksaan Akun Aktif (Baris 165–167):

Memeriksa kondisi apakah akunAktif == null.

Jika belum ada akun yang dibuat atau dipilih, sistem mencetak pesan peringatan "Peringatan: Pilih atau buat akun terlebih dahulu!" dan proses tidak dilanjutkan.

Pemeriksaan Tipe Objek dengan instanceof (Baris 168–169):

Menguji apakah akun yang sedang aktif (akunAktif) merupakan instansi dari kelas RekeningTabungan menggunakan kata kunci instanceof.

Downcasting dan Eksekusi Method Spesifik (Baris 170–172):

Jika Tipe Sesuai (RekeningTabungan): Melakukan downcasting dengan mengubah tipe data akunAktif (yang bertipe umum Rekening) menjadi RekeningTabungan melalui frasa (RekeningTabungan) akunAktif.

Setelah di-cast, program memanggil method khusus tabungan.tambahBungaAkhirBulan() untuk memproses perhitungan bunga akhir bulan.

Penanganan Tipe Akun Tidak Sesuai (Baris 173–175):

Jika Bukan RekeningTabungan: Menjalankan blok else dan menampilkan pesan kesalahan "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan." (misalnya saat akun yang aktif adalah RekeningGiro).

```

163 // case 7: Simulasi Waktu Akhir Bulan (Tuas 2)
164 // if (akunAktif == null) {
165 //     System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
166 // } else {
167 //     // 2. Memeriksa tipe objek dengan instanceof
168 //     if (akunAktif instanceof RekeningTabungan) {
169 //         // 3. Downcasting ke tipe RekeningTabungan & memanggil method
170 //         RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
171 //         tabungan.tambahBungaAkhirBulan();
172 //     } else {
173 //         // 4. Pesan penolakan jika bukan RekeningTabungan
174 //         System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
175 //     }
176 // }
177 break;
178

```

g. Case 0: Keluar Program

Menghentikan loop do-while dan menutup objek scanner (scanner.close()) untuk membebaskan sumber daya memori.

```

180 // case 0:
181 // System.out.println("Terima kasih telah menggunakan sistem perbankan.");
182 // break;
183 // default:
184 // System.out.println("Pilihan tidak valid. Silakan coba lagi.");
185 // }
186 // } while (pilihan != 0);
187 // scanner.close();
188 // }
189 // }
190 // }
191 // }

```

4. Kode Secara Lengkap

```

package Pekan4_2511533025;

import java.util.ArrayList;
import java.util.Scanner;

public class MainPerbankan {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new
        ArrayList<>();
        Rekening akunAktif = null;
        int pilihan;

        do {
            System.out.println("\n=== SISTEM PERBANKAN
            MINI (MULTI-AKUN) ===");
            System.out.println("-----
            -----");
            if (akunAktif == null) {
                System.out.println("Akun Aktif : Tidak ada
                akun yang dipilih");
            } else {

```

```

        System.out.println("Akun Aktif : " +
akunAktif.getNamaPemilik() + " (" +
akunAktif.getNomorRekening() + ")");
    }
    System.out.println("-----
-----");
    System.out.println("Menu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi
Rekening");
    System.out.println("5. Ganti Akun (Pilih
Akun)");
    System.out.println("6. Tampilkan Riwayat
Transaksi");
    System.out.println("7. Simulasi Akhir Bulan
(Khusus Tabungan)");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

    pilihan = scanner.nextInt();
    scanner.nextLine(); // Clear buffer

    switch (pilihan) {
        case 1:
            // Modifikasi Case 1: Buka Rekening
            Baru (Tugas 1)
            System.out.print("Masukkan No.
Rekening: ");

            String no = scanner.nextLine();
            System.out.print("Masukkan Nama
Pemilik: ");

            String nama = scanner.nextLine();
            System.out.print("Masukkan Saldo Awal:
");

            double saldoAwal =
scanner.nextDouble();
            scanner.nextLine(); // Clear buffer
            System.out.print("Buat PIN (6 digit):
");

            String pin = scanner.nextLine();

            // 1. Opsi Produk
            System.out.println("Pilih Produk: 1.
Tabungan Umum | 2. Giro Bisnis");
            System.out.print("Pilihan Anda: ");
            int jenisProduk = scanner.nextInt();
            scanner.nextLine(); // Clear buffer

            Rekening rekBaru = null;

            if (jenisProduk == 1) {
                // 2. Instansiasi RekeningTabungan

```

```

        System.out.print("Masukkan Suku
Bunga (%): ");
        scanner.nextDouble();
        double sukuBunga =
        scanner.nextLine();
        rekBaru = new RekeningTabungan(no,
nama, saldoAwal, pin, sukuBunga);
    } else if (jenisProduk == 2) {
        // 3. Instansiasi RekeningGiro
        System.out.print("Masukkan Batas
Overdraft (Limit Pinjaman): ");
        scanner.nextDouble();
        double batasOverdraft =
        scanner.nextLine();
        rekBaru = new RekeningGiro(no,
nama, saldoAwal, pin, batasOverdraft);
    } else {
        System.out.println("Pilihan produk
tidak valid! Pembuatan rekening dibatalkan.");
        break;
    }

    // 4. Upcasting: Menyimpan ke
    daftarRekening bertipe ArrayList<Rekening>
    daftarRekening.add(rekBaru);
    akunAktif = rekBaru; // Otomatis
    jadikan akun aktif
    break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Peringatan:
Pilih atau buat akun terlebih dahulu!");
        } else {
            System.out.print("Masukkan nominal
setor tunai: ");
            scanner.nextDouble();
            double setor =
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Peringatan:
Pilih atau buat akun terlebih dahulu!");
        } else {
            System.out.print("Masukkan PIN
Anda: ");
            scanner.nextLine();
            String inputPin =
            if
            (akunAktif.otentikasi(inputPin)) {
                System.out.print("Masukkan
nominal tarik tunai: ");
            }
        }
    }
}

```

```

                                double tarik =
scanner.nextDouble();
                                akunAktif.tarikTunai(tarik);
                                } else {
                                System.out.println("Transaksi
Gagal: PIN salah!");
                                }
                                }
                                break;

                                case 4:
                                if (akunAktif == null) {
                                System.out.println("Peringatan:
Pilih atau buat akun terlebih dahulu!");
                                } else {
                                akunAktif.cekInformasi();
                                }
                                break;

                                case 5:
                                if (daftarRekening.isEmpty()) {
                                System.out.println("Belum ada
rekening yang terdaftar.");
                                } else {
                                System.out.println("---- DAFTAR
AKUN ----");
                                for (int i = 0; i <
daftarRekening.size(); i++) {
                                System.out.println((i + 1) +
". " + daftarRekening.get(i).getNamaPemilik() + " (" +
daftarRekening.get(i).getNomorRekening() + ")");
                                }
                                System.out.print("Pilih nomor
akun: ");
                                int indeks = scanner.nextInt() -
1;
                                scanner.nextLine(); // Clear
buffer

                                if (indeks >= 0 && indeks <
daftarRekening.size()) {
                                Rekening target =
daftarRekening.get(indeks);
                                System.out.print("Masukkan PIN
akun target: ");
                                String pinAkses =
scanner.nextLine();

                                if
(target.otentikasi(pinAkses)) {
                                akunAktif = target;

                                System.out.println("Berhasil beralih ke akun " +
akunAktif.getNamaPemilik());
                                } else {

```

```

        System.out.println("Gagal
beralih: PIN salah!");
    }
    } else {
        System.out.println("Pilihan
tidak valid.");
    }
}
break;

case 6:
    if (akunAktif == null) {
        System.out.println("Peringatan:
Pilih atau buat akun terlebih dahulu!");
    } else {
        System.out.print("Masukkan PIN
Anda: ");
        String inputPin =
scanner.nextLine();
        if
(akunAktif.otentikasi(inputPin)) {
            if
(akunAktif.getRiwayatTransaksi().isEmpty()) {
                System.out.println("Belum
ada riwayat transaksi pada akun ini.");
            } else {
                System.out.println("----
RIWAYAT TRANSAKSI ----");
                for (Transaksi trx :
akunAktif.getRiwayatTransaksi()) {
                    System.out.println(trx);
                }
                System.out.println("-----
-----");
            }
        } else {
            System.out.println("Akses
Ditolak: PIN salah!");
        }
    }
    break;

case 7:
    // Simulasi Waktu Akhir Bulan (Tugas
2)
    if (akunAktif == null) {
        System.out.println("Peringatan:
Pilih atau buat akun terlebih dahulu!");
    } else {
        // 2. Memeriksa tipe objek dengan
instanceof
        if (akunAktif instanceof
RekeningTabungan) {

```

```

// 3. Downcasting ke tipe
RekeningTabungan & memanggil method
RekeningTabungan tabungan =
(RekeningTabungan) akunAktif;

tabungan.tambahBungaAkhirBulan();
    } else {
// 4. Pesan penolakan jika
bukan RekeningTabungan
        System.out.println("Gagal:
Fitur bunga akhir bulan hanya berlaku untuk Rekening
Tabungan.");
    }
    }
    break;

    case 0:
        System.out.println("Terima kasih telah
menggunakan sistem perbankan.");
        break;

    default:
        System.out.println("Pilihan tidak
valid. Silakan coba lagi.");
    }
    } while (pilihan != 0);

    scanner.close();
}
}

```

d. Class RekeningTabungan

1. Deklarasi Class Subclass dan Atribut Spesifik

Pendeklarasian Package (Baris 1):

package Pekan4_2511533025; menentukan lokasi package atau direktori tempat file kelas ini disimpan.

Deklarasi Class & Inheritance/Pewarisan (Baris 3):

public class RekeningTabungan extends Rekening mendefinisikan kelas RekeningTabungan sebagai anak kelas (subclass) dari kelas induk Rekening dengan kata kunci extends. Artinya, RekeningTabungan mewarisi semua atribut dan method yang ada pada kelas Rekening.

Penambahan Atribut Spesifik Subclass (Baris 5–6):

private double sukuBunga; menambah atribut baru khusus untuk RekeningTabungan bertipe double dengan hak akses private

(enkapsulasi). Atribut ini digunakan untuk menyimpan persentase bunga tabungan yang tidak dimiliki oleh kelas induk Rekening biasa.

```
1 package Pekan4_2511533025;
2
3 public class RekeningTabungan extends Rekening {
4
5     // Atribut spesifik yang hanya dimiliki oleh Tabungan
6     private double sukuBunga;
7 }
```

2. Constructor Subclass

Deklarasi Constructor Subclass (Baris 9):

`public RekeningTabungan(...)` mendefinisikan constructor untuk kelas `RekeningTabungan` yang menerima 5 parameter input: nomor, nama, saldoAwal, pinAwal, dan sukuBunga.

Pemanggilan Constructor Kelas Induk (Baris 10–11):

`super(nomor, nama, saldoAwal, pinAwal);` dipanggil untuk meneruskan 4 nilai awal ke constructor milik kelas induk (`Rekening`). Pemanggilan `super()` ini wajib ditulis di baris pertama di dalam constructor subclass.

Inisialisasi Atribut Spesifik (Baris 12):

`this.sukuBunga = sukuBunga;` menyimpan nilai dari parameter `sukuBunga` ke dalam atribut `sukuBunga` milik objek `RekeningTabungan` itu sendiri.

```
8 // Constructor Subclass
9 public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
10     // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
11     super(nomor, nama, saldoAwal, pinAwal);
12     this.sukuBunga = sukuBunga;
13 }
14 }
```

3. Method Tambah Bunga Akhir Bulan

Deklarasi Method (Baris 15):

`pSystem.out.println(tambahBungaAkhirBulan())` mendefinisikan method bertipe `void` (tidak mengembalikan nilai) yang digunakan khusus untuk memproses penambahan bunga di akhir bulan pada `RekeningTabungan`.

Perhitungan dan Penambahan Bunga (Baris 19–20):

`double nominalBunga = saldo * (sukuBunga / 100);` menghitung besaran nominal bunga berdasarkan saldo saat ini dan persentase suku bunga. Atribut `saldo` dari kelas induk `Rekening` bisa diakses langsung karena menggunakan modifier `protected`.

saldo += nominalBunga; menambahkan hasil perhitungan nominal bunga tersebut ke saldo rekening.

Mencatat Riwayat Transaksi (Baris 23–24):

String idTrx = "TRX-B-" + System.currentTimeMillis(); membuat ID transaksi unik dengan gabungan teks "TRX-B-" dan timestamp sistem saat itu.

riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga)); membuat objek Transaksi baru berkategori "Bunga" lalu menyimpannya ke dalam ArrayList riwayatTransaksi.

Output Informasi (Baris 26):

System.out.println(...) mencetak pesan ke layar konsol yang mengonfirmasi bahwa bunga sebesar persentase sukuBunga% telah berhasil ditambahkan ke rekening beserta nilai rupiahnya.

```
15 public void tambahBungaAkhirBulan() {
16     // Menghitung bunga
17     // Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
18     // Karena atribut 'saldo' di class Rekening menggunakan modifier 'protected'
19     double nominalBunga = saldo * (sukuBunga / 100);
20     saldo += nominalBunga;
21
22     // Mencatat riwayat transaksi
23     String idTrx = "TRX-B-" + System.currentTimeMillis();
24     riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
25
26     System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
27 }
28 }
```

e. Class RekeningGiro

1. Deklarasi Class Subclass dan Atribut Spesifik

Pendeklarasian Package (Baris 1):

package Pekan4_2511533025; menentukan package tempat file kelacerukan berada.

Deklarasi Class & Inheritance/Pewarisan (Baris 3):

public class RekeningGiro extends Rekening mendefinisikan kelas RekeningGiro sebagai anak kelas (subclass) dari kelas induk Rekening dengan kata kunci extends. Artinya, RekeningGiro otomatis mewarisi seluruh atribut dan method dari Rekening.

Penambahan Atribut Spesifik Subclass (Baris 5):

private double batasOverdraft; menambahkan atribut khusus untuk RekeningGiro bertipe double dengan akses private (enkapsulasi).

Atribut ini dipakai untuk menyimpan batas maksimal fasilitas cerukan atau limit pinjaman saldo yang diizinkan untuk akun giro.

```
1 package Pekan4_2511533025;
2
3 public class RekeningGiro extends Rekening {
4
5     private double batasOverdraft;
6 }
```

2. Constructor Subclass

Deklarasi Constructor Subclass (Baris 7):

`public RekeningGiro(...)` mendefinisikan constructor untuk kelas `RekeningGiro` yang menerima 5 parameter input: nomor, nama, saldoAwal, pinAwal, dan batasOverdraft.

Pemanggilan Constructor Kelas Induk (Baris 8–9):

`super(nomor, nama, saldoAwal, pinAwal);` dipanggil untuk meneruskan 4 nilai awal ke constructor milik kelas induk (`Rekening`). Pemanggilan `super()` ini wajib ditulis di baris pertama di dalam constructor subclass.

Inisialisasi Atribut Spesifik (Baris 10):

`this.batasOverdraft = batasOverdraft;` menyimpan nilai dari parameter `batasOverdraft` ke dalam atribut `batasOverdraft` milik objek `RekeningGiro` itu sendiri.

```
7 public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
8     // Memanggil inisialisasi dasar dari Superclass
9     super(nomor, nama, saldoAwal, pinAwal);
10    this.batasOverdraft = batasOverdraft;
11 }
```

3. Getter khusus Giro

Deklarasi Method Getter (Baris 14):

`public double getBatasOverdraft()` mendefinisikan sebuah method akses (getter) publik yang mengembalikan nilai bertipe `double`. Method ini khusus digunakan untuk mengambil nilai batas overdraft (limit pinjaman).

Pengembalian Nilai Atribut (Baris 15):

`return batasOverdraft;` mengembalikan nilai dari atribut `batasOverdraft` yang berstatus `private` agar dapat dibaca dari luar kelas secara aman tanpa mengubah nilainya.

```
13 // Getter khusus Giro
14 public double getBatasOverdraft() {
15     return batasOverdraft;
16 }
17
18 // (Catatan: Penarikan hingga limit overdraft akan kita selesaikan di Modul 5)
19 }
```

d. Gambar Ouput Kode Program

Untuk outputnya sama dengan output tugas atau Latihan praktikum, karena saling berhubungan dibagian class mainnya. Jadi gambar outputnya ada di bagian tugas praktikum berikut ini.

B. TUGAS PRAKTIKUM

a. Pembuatan Multi-Produk

1. Modifikasi case 1: Buka Rekening Baru di Main.java.
 - Setelah user mSuperclass-nyaldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
 - Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
 - Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
 - Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

b. Simulasi Waktu Akhir Bulan

- c. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
- d. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
- e. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- f. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

Berdasarkan tugas yang diberikan, cukup mengubah kode yang ada di clas MainPerbankan:

1. Class MainPerbankan
 - a) Pada tugas satu ini masih menggunakan kode program praktikum yaitu class Transaksi, Rekening, RekeningTabungan, RekeningGiro, dan kode yang ada di class MainPerbankan.

Berikut kode yang berubah:

1. Menambahkan pilihan menu yang awalnya 7 menjadi 8, yaitu menu ke 7 Simulasi Akhir Bulan

```

22 System.out.println("Menu Utama:");
23 System.out.println("1. Buka Rekening Baru");
24 System.out.println("2. Setor Tunai");
25 System.out.println("3. Tarik Tunai");
26 System.out.println("4. Cek Informasi Rekening");
27 System.out.println("5. Ganti Akun (Pilih Akun)");
28 System.out.println("6. Tampilkan Riwayat Transaksi");
29 System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
30 System.out.println("0. Keluar");
31 System.out.print("Pilih menu: ");
32

```

2. Perubahan case 1 (Pembuatan Rekening Baru & Upcasting), yaitu menambahkan submenu pilihan jenis produk (1. Tabungan Umum / 2. Giro Bisnis). Membuat objek subclass RekeningTabungan atau RekeningGiro lalu menyimpannya ke dalam ArrayList<Rekening> (Upcasting).

Input Data Nasabah (Baris 38–46):

Meminta input dasar berupa nomor rrrekBaru nama pemilik, saldo awal, dan PIN dari pengguna menggunakan Scanner. scanner.nextLine() di baris 44 digunakan untuk membersihkan buffer dari sisa baris baru setelah nextDouble().

Pemilihan Jenis Produk (Baris 48–52):

Pengguna diminta memilih produk: 1 untuk Tabungan Umum atau 2 untuk Giro Bisnis.

Constructor & Instansiasi (Baris 54–71):

Rekening rekBaru = null; disiapkan sebagai variabel referensi kelas induk (Rekening).

Opsi 1 (RekeningTabungan): Meminta input suku bunga, lalu memanggil constructor new RekeningTabungan(...).

Opsi 2 (RekeningGiro): Meminta input batas overdraft (limit pinjaman), lalu memanggil constructor new RekeningGiro(...).

Jika pilihan tidak valid, program menampilkan pesan kesalahan dan menghentikan proses pembuatan rekening dengan break;.

Upcasting & Penyimpanan Struktur Data (Baris 73–75):

Upcasting: Objek RekeningTabungan atau RekeningGiro disimpan ke dalam daftarRekening bertipe ArrayList<Rekening>. Kelas turunan secara otomatis dikonversi menjadi tipe kelas induknya (Rekening).

Pengeset Akun Aktif: `akunAktif = rekBaru`; secara otomatis menetapkan rekening yang baru dibuat sebagai akun yang sedang aktif digunakan di dalam sistem.

```

36 case 1:
37 // Modifikasi Case 1: Buka Rekening Baru (Tugas 1)
38 System.out.print("Masukkan No. Rekening: ");
39 String no = scanner.nextLine();
40 System.out.print("Masukkan Nama Pemilik: ");
41 String nama = scanner.nextLine();
42 System.out.print("Masukkan Saldo Awal: ");
43 double saldoAwal = scanner.nextDouble();
44 scanner.nextLine(); // Clear buffer
45 System.out.print("Buat PIN (6 digit): ");
46 String pin = scanner.nextLine();
47
48 // 1. Opsi Produk
49 System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
50 System.out.print("Pilihan Anda: ");
51 int jenisProduk = scanner.nextInt();
52 scanner.nextLine(); // Clear buffer
53
54 Rekening rekBaru = null;
55
56 if (jenisProduk == 1) {
57 // 2. Instansiasi RekeningTabungan
58 System.out.print("Masukkan Suku Bunga (%): ");
59 double sukuBunga = scanner.nextDouble();
60 scanner.nextLine();
61 rekBaru = new RekeningTabungan(no, nama, saldoAwal, pin, sukuBunga);
62 } else if (jenisProduk == 2) {
63 // 3. Instansiasi RekeningGiro
64 System.out.print("Masukkan Batas Overdraft (Limit Pinjaman): ");
65 double batasOverdraft = scanner.nextDouble();
66 scanner.nextLine();
67 rekBaru = new RekeningGiro(no, nama, saldoAwal, pin, batasOverdraft);
68 } else {
69 System.out.println("Pilihan produk tidak valid! Pembuatan rekening dibatalkan.");
70 break;
71 }
72
73 // 4. Upcasting: Menyimpan ke daftarRekening bertipe ArrayList<Rekening>
74 daftarRekening.add(rekBaru);
75 akunAktif = rekBaru; // Otomatis jadikan akun aktif
76 break;
77

```

3. Penambahan case 7, yaitu menambahkan opsi menu ke-7. Di dalam case 7, dilakukan pengecekan apakah `akunAktif` merupakan objek dari `RekeningTabungan` menggunakan keyword `instanceof`, kemudian dilakukan Downcasting (`RekeningTabungan`) `akunAktif` untuk memanggil method `tambahBungaAkhirBulan()`.

Pemeriksaan Akun Aktif (Baris 165–167):

Memeriksa kondisi apakah `akunAktif == null`.

Jika belum ada akun yang dibuat atau dipilih, sistem mencetak pesan peringatan "Peringatan: Pilih atau buat akun terlebih dahulu!" dan proses tidak dilanjutkan.

Pemeriksaan Tipe Objek dengan `instanceof` (Baris 168–169):

Menguji apakah akun yang sedang aktif (`akunAktif`) merupakan instansi dari kelas `RekeningTabungan` menggunakan kata kunci `instanceof`.

Downcasting dan Eksekusi Method Spesifik (Baris 170–172):

Jika Tipe Sesuai (`RekeningTabungan`): Melakukan downcasting dengan mengubah tipe data `akunAktif` (yang bertipe umum

Rekening) menjadi RekeningTabungan melalui frasa (RekeningTabungan) akunAktif.

Setelah di-cast, program memanggil method khusus tabungan.tambahBungaAkhirBulan() untuk memproses perhitungan bunga akhir bulan.

Penanganan Tipe Akun Tidak Sesuai (Baris 173–175):

Jika Bukan RekeningTabungan: Menjalankan blok else dan menampilkan pesan kesalahan "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan." (misalnya saat akun yang aktif adalah RekeningGiro).

```
163 // case 7:
164 // Simulasi Waktu Akhir Bulan (Tugas 2)
165 if (akunAktif == null) {
166     System.out.println("Peringatan: Pilih atau buat akun terlebih dahulu!");
167 } else {
168     // 2. Periksa tipe objek dengan instanceof
169     if (akunAktif instanceof RekeningTabungan) {
170         // 3. Downcasting ke tipe RekeningTabungan & memanggil method
171         RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
172         tabungan.tambahBungaAkhirBulan();
173     } else {
174         // 4. Pesan penolakan jika bukan RekeningTabungan
175         System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
176     }
177 }
178 break;
```

b) Kode Program Secara Lengkap Dari Class MainPerbankan

1. Class MainPerbankan

```
package Pekan4_2511533025;

import java.util.ArrayList;
import java.util.Scanner;
public class MainPerbankan {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new
        ArrayList<>();
        Rekening akunAktif = null;
        int pilihan;

        do {
            System.out.println("\n=== SISTEM
PERBANKAN MINI (MULTI-AKUN) ===");
            System.out.println("-----
-----");
            if (akunAktif == null) {
                System.out.println("Akun Aktif :
Tidak ada akun yang dipilih");
            } else {
```

```

        System.out.println("Akun Aktif : " +
akunAktif.getNamaPemilik() + " (" +
akunAktif.getNomorRekening() + ")");
    }
    System.out.println("-----
-----");
    System.out.println("Menu Utama:");
    System.out.println("1. Buka Rekening
Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi
Rekening");
    System.out.println("5. Ganti Akun (Pilih
Akun)");
    System.out.println("6. Tampilkan Riwayat
Transaksi");
    System.out.println("7. Simulasi Akhir
Bulan (Khusus Tabungan)");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

    pilihan = scanner.nextInt();
    scanner.nextLine(); // Clear buffer

    switch (pilihan) {
        case 1:
            // Modifikasi Case 1: Buka
Rekening Baru (Tugas 1)
            System.out.print("Masukkan No.
Rekening: ");

            String no = scanner.nextLine();
            System.out.print("Masukkan Nama
Pemilik: ");

            String nama =
scanner.nextLine();
            System.out.print("Masukkan Saldo
Awal: ");

            double saldoAwal =
scanner.nextDouble();
            scanner.nextLine(); // Clear
buffer
            System.out.print("Buat PIN (6
digit): ");

            String pin = scanner.nextLine();

            // 1. Opsi Produk
            System.out.println("Pilih
Produk: 1. Tabungan Umum | 2. Giro Bisnis");
            System.out.print("Pilihan Anda:
");

            int jenisProduk =
scanner.nextInt();
            scanner.nextLine(); // Clear
buffer

```

```

        Rekening rekBaru = null;

        if (jenisProduk == 1) {
            // 2. Instansiasi
            RekeningTabungan
            Suku Bunga (%): ";
            scanner.nextDouble();
            RekeningTabungan(no, nama, saldoAwal, pin,
            sukuBunga);
            System.out.print("Masukkan
            double sukuBunga =
            scanner.nextLine();
            rekBaru = new
        } else if (jenisProduk == 2) {
            // 3. Instansiasi
            RekeningGiro
            System.out.print("Masukkan
            Batas Overdraft (Limit Pinjaman): ");
            double batasOverdraft =
            scanner.nextDouble();
            scanner.nextLine();
            rekBaru = new
            RekeningGiro(no, nama, saldoAwal, pin,
            batasOverdraft);
        } else {
            System.out.println("Pilihan
            produk tidak valid! Pembuatan rekening
            dibatalkan.");
            break;
        }

        // 4. Upcasting: Menyimpan ke
        daftarRekening bertipe ArrayList<Rekening>
        daftarRekening.add(rekBaru);
        akunAktif = rekBaru; // Otomatis
        jadikan akun aktif
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Peringatan: Pilih atau buat akun
            terlebih dahulu!");
        } else {
            System.out.print("Masukkan
            nominal setor tunai: ");
            double setor =
            scanner.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {

```

```

System.out.println("Peringatan: Pilih atau buat akun
terlebih dahulu!");
    } else {
        System.out.print("Masukkan
PIN Anda: ");
        String inputPin =
scanner.nextLine();
        if
(akunAktif.otentikasi(inputPin)) {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik =
scanner.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Transaksi Gagal: PIN salah!");
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Peringatan: Pilih atau buat akun
terlebih dahulu!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum
ada rekening yang terdaftar.");
        } else {
            System.out.println("----
DAFTAR AKUN ----");
            for (int i = 0; i <
daftarRekening.size(); i++) {
                System.out.println((i +
1) + ". " + daftarRekening.get(i).getNamaPemilik() +
" (" + daftarRekening.get(i).getNomorRekening() +
")");
            }
            System.out.print("Pilih
nomor akun: ");
            int indeks =
scanner.nextInt() - 1;
            scanner.nextLine(); // Clear
buffer

```

```

                                if (indeks >= 0 && indeks <
daftarRekening.size()) {
                                Rekening target =
daftarRekening.get(indeks);
System.out.print("Masukkan PIN akun target: ");
                                String pinAkses =
scanner.nextLine();

                                if
(target.otentikasi(pinAkses)) {
                                    akunAktif = target;

System.out.println("Berhasil beralih ke akun " +
akunAktif.getNamaPemilik());
                                } else {

System.out.println("Gagal beralih: PIN salah!");
                                }
                                } else {

System.out.println("Pilihan tidak valid.");
                                }
                                }
                                break;

                                case 6:
                                    if (akunAktif == null) {

System.out.println("Peringatan: Pilih atau buat akun
terlebih dahulu!");
                                    } else {
                                        System.out.print("Masukkan
PIN Anda: ");
                                        String inputPin =
scanner.nextLine();
                                        if
(akunAktif.otentikasi(inputPin)) {
                                            if
(akunAktif.getRiwayatTransaksi().isEmpty()) {

System.out.println("Belum ada riwayat transaksi pada
akun ini.");
                                            } else {

System.out.println("---- RIWAYAT TRANSAKSI ----");
                                                for (Transaksi trx :
akunAktif.getRiwayatTransaksi()) {

System.out.println(trx);
                                                }

System.out.println("-----
--");
                                            }
                                        }
                                    }
                                }

```

```

        } else {
System.out.println("Akses Ditolak: PIN salah!");
        }
        break;
    case 7:
        // Simulasi Waktu Akhir Bulan
        (Tugas 2)
        if (akunAktif == null) {
System.out.println("Peringatan: Pilih atau buat akun
terlebih dahulu!");
        } else {
            // 2. Memeriksa tipe objek
            dengan instanceof
            if (akunAktif instanceof
RekeningTabungan) {
                // 3. Downcasting ke
                tipe RekeningTabungan & memanggil method
                RekeningTabungan
                tabungan = (RekeningTabungan) akunAktif;
                tabungan.tambahBungaAkhirBulan();
            } else {
                // 4. Pesan penolakan
                jika bukan RekeningTabungan
System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan.");
            }
        }
        break;
    case 0:
        System.out.println("Terima kasih
telah menggunakan sistem perbankan.");
        break;
    default:
        System.out.println("Pilihan
tidak valid. Silakan coba lagi.");
    }
    } while (pilihan != 0);
    scanner.close();
}
}

```

c) Gambar Output Program Praktikum dan Tugas

Kode diatas ketika dijalankan akan mengeluarkan menu, dimana menu terdiri dari 8 pilihan, untuk informasi lebih lanjut bisa dilihat dari gambar secreeennshot kode tersebut ketika dijalankan:

```
=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Tidak ada akun yang dipilih
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No. Rekening: 2511533025
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 500000
Buat PIN (6 digit): 013579
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan Anda: 1
Masukkan Suku Bunga (%): 2,5
Rekening atas nama Dika berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Dika (2511533025)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 2
Masukkan nominal setor tunai: 500000
Setor tunai Rp500000.0 berhasil. Saldo saat ini: Rp1000000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Dika (2511533025)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 2.5% berhasil ditambahkan: Rp25000.0
```

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

Akun Aktif : Dika (2511533025)

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 4

--- INFO REKENING ---

No. Rekening : 2511533025

Nama Pemilik : Dika

Saldo Akhir : Rp1025000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

Akun Aktif : Dika (2511533025)

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 3

Masukkan PIN Anda: 013579

Masukkan nominal tarik tunai: 25000

Tarik tunai Rp25000.0 berhasil. Saldo saat ini: Rp1000000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

Akun Aktif : Dika (2511533025)

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 6

Masukkan PIN Anda: 013579

--- RIWAYAT TRANSAKSI ---

ID: TRX-S-1790768769905 | Jenis: Kredit | Nominal: Rp500000.0

ID: TRX-B-1790768773495 | Jenis: Bunga | Nominal: Rp25000.0

ID: TRX-T-1790768801596 | Jenis: Debit | Nominal: Rp25000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

Akun Aktif : Dika (2511533025)

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 1

Masukkan No. Rekening: 2511532017

Masukkan Nama Pemilik: Arif

Masukkan Saldo Awal: 300000

Buat PIN (6 digit): 024680

Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis

Pilihan Anda: 2

Masukkan Batas Overdraft (Limit Pinjaman): 200000

Rekening atas nama Arif berhasil dibuat.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===

Akun Aktif : Arif (2511532017)

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 4

--- INFO REKENING ---

No. Rekening : 2511532017

Nama Pemilik : Arif

Saldo Akhir : Rp300000.0

```

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Arif (2511532017)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Arif (2511532017)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 2
Masukkan nominal setor tunai: 200000
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp500000.0

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Arif (2511532017)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 6
Masukkan PIN Anda: 024680
--- RIWAYAT TRANSAKSI ---
ID: TRX-S-1790769972334 | Jenis: Kredit | Nominal: Rp200000.0
-----

=== SISTEM PERBANKAN MINI (MULTI-AKUN) ===
-----
Akun Aktif : Arif (2511532017)
-----
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun (Pilih Akun)
6. Tampilkan Riwayat Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 0
Terima kasih telah menggunakan sistem perbankan.

```

C. Link Repository GitHub

https://github.com/DikaSiNpc/PrakPBO_C_2511533025/tree/PrakPBO_Pekan4_C_2511533025/src